

SD TCL - Programs and the Catalogue

Compiling SD BASIC, running it, and giving it a name you can type.

Getting from source to something you can type is three steps: **compile it, put it in a catalogue, then call it by name**. Only the first is compulsory.

SD folds case, so a command may be typed in either case. Commands are shown here in lower case. In the tables, *italics* mark something you supply and **bold** marks a word typed as it stands; braces mark an optional part.

Every listing on this page was produced by running it, on SD Core for Windows W1.0-0, against a two-line program in an account's `bp` file.

Compiling

```
basic {file} record
```

The file defaults to `bp`. What comes back names the file and record, then the error count:

```
basic bp zzprog
```

```
Compiling bp zzprog
WARNING: Final END statement is missing
0 error(s)
Compiled 1 program(s) with no errors
```

A warning is not an error and the program still runs. The line above is a warning, the count says `0 error(s)`, and that program ran. **Read the count, not the presence of output.**

`@system.return.code` is the number of programs compiled, and negative on a fatal error.

The object code goes into `bp.out`, a separate file beside `bp`, one record per program. **Its name is lower case even when the source record is not** — a source record `ZZPROG` compiles to an object record `zzprog`. That is this port's naming, and it is worth knowing before you go looking for the object by the name you typed.

Running without cataloguing

```
run {file} record
```

```
run bp zzprog
```

```
ZZPROG OK
```

`run` needs the file and record name every time. It is what you use while you are still writing the thing.

The catalogue is what gives a program a name

```
catalog {file {call.name}} {program | *} {local | global | pcode} {no.xref}
```

Catalogue it and the name becomes a command:

```
catalog bp zzprog  
zzprog hello
```

```
ZZPROG added to private catalogue  
ZZPROG OK
```

The program sees the whole command line. `@sentence` in the program above read `zzprog hello`, so arguments arrive as typed and the program parses them itself.

THREE CATALOGUES, AND THEY ARE NOT THE SAME

private	the account's own. The default when no keyword is given
----------------	---

local	a VOC entry in the account, so the name works only there
--------------	--

global	<code>gcat</code> in the system account — every account sees it
---------------	--

Global cataloguing requires administrator privilege, and on this port that means an elevated session. Without it you get *"Command requires administrator privileges"*. **The same gate applies to an implicit global catalogue** — one chosen by a `*`, `!`, `_` or `$` prefix on the call name rather than by the `global` keyword — so the prefix is not a way round it.

Private and local cataloguing need none of this and work in a programmer's own account.

SEEING AND REMOVING

```
map {all} {lptr {n}} {file {name}}
```

```
System catalogue map at 23:12:07 on 26 Aug 2026                Page 1
Catalogue name..... Compiled.  Time....  ...Obj  ..Xref  ..Size
ZZPROG                26 Aug 26  23:12:07    189      6    195
                        Total:      195
```

`Obj` and `Xref` are the object-code and cross-reference sizes in bytes, and `Size` is the total. An empty one says so:

```
Catalogue is empty
```

```
delete.catalog name... {global | local}
```

```
ZZPROG deleted from the private catalogue
```

`map` **also takes a `file` option**, which writes the map to a file instead of the screen. **It asks before overwriting** — *"File xx will be cleared and overwritten. Continue (Y/N)?"* — so it is not usable from a script driving SD down a pipe.

Formatting source

```
format {file} {record} {case}
```

`format` rewrites a BASIC source record with standard indentation. **It says nothing at all when it succeeds** — no output is the success case, which is worth knowing before you go looking for a confirmation that never comes. `case` additionally applies case conversion.

With no record name it uses the active select list.

Compiling dictionaries

```
cd {dict | data} file {i-type}... {no.query} {no.page}
cd local
cd all
```

`compile.dict` is the same verb under its longer name. It compiles the A, C, I and S-type entries in a dictionary; with no names it does all of them, or uses the active select list if there is one.

<code>cd local</code>	every dictionary in the account
-----------------------	---------------------------------

<code>cd all</code>	every dictionary visible from the account
---------------------	---

Compiling a dictionary is a performance step, not a correctness one: it is what a query using those entries runs against.

Named common blocks

```
list.common {all}
delete.common name
```

```
No named common blocks present
```

Named common survives between programs in a session, and `list.common` is how you see what is holding memory. `all` additionally shows the system blocks and is internal-only.

The two that are not like the others

`generate` builds `$include` token definitions from a dictionary. **It is interactive** — it asks for the token type, the prefix and several other things — so it cannot be driven from a script.

`debug` runs a program under the debugger, taking the same arguments as `run`. It needs a terminal and **refuses inside a phantom** with *"Cannot debug program in a phantom process"*. The debugger itself is covered in SD Basic - Debugging.

Who has these verbs

standard

`format` `list.common`

programmer

`basic` `catalog` `catalogue` `cd` `compile.dict` `debug` `delete.catalog`
`delete.catalogue` `delete.common` `generate` `map` `run`

A standard account can run catalogued programs and cannot make them. That is the whole point of the split: an application is deployed by cataloguing it, and the people who use the application need none of the verbs on this page to run it.

`format` and `list.common` are the two exceptions, and both are harmless — one tidies source the account can already read, the other reports on the session's own memory.

See also

SD Basic - Program Structure · SD Basic - Debugging · SD TCL - Files and Records.